

The Complete Ansible Guide

Why Ansible • Command Basics • Real Interview Questions & Answers

Companion guide to the Ansible-Learning-Hub repository

1. Why Ansible?

Ansible is an open-source automation tool used for configuration management, application deployment, and orchestration. It lets you describe infrastructure and application state as code (YAML) and applies that state consistently, repeatedly, and predictably across any number of servers.

What makes it different

Feature	Ansible	Puppet / Chef	SaltStack
Agent required?	No (agentless, uses SSH)	Yes (agent on every node)	Optional (agent or agentless)
Language	YAML (easy to read)	Ruby-based DSL	YAML / Python
Learning curve	Low	Moderate–High	Moderate
Push / Pull	Push-based	Pull-based (agent polls)	Both
Setup complexity	Minimal	Higher (master + agents)	Higher

Core benefits

- **Agentless** — connects over standard SSH (or WinRM for Windows); nothing to install on managed nodes.
- **Idempotent** — running the same playbook twice produces the same end state, with no unintended side effects.
- **Simple, human-readable YAML** — playbooks read almost like documentation.
- **Huge module ecosystem** — thousands of built-in modules for cloud, network, containers, databases, and more.
- **Orchestration** — can sequence multi-tier rollouts (e.g. database before app servers) in one run.
- **Strong community & enterprise backing** — maintained by Red Hat, widely adopted in DevOps pipelines.

Where it fits in a DevOps pipeline

Ansible typically sits in the **configuration management and deployment** stage: after code is built and tested (CI), and after infrastructure is provisioned (e.g. by Terraform), Ansible installs packages, manages configuration files, and deploys the application onto the target servers (CD). See the three flow diagrams in the accompanying repository for the exact hand-off points.

2. Ansible Command Basics

2.1 Installation & setup

```
pip install ansible
ansible --version
ansible-config init --disabled > ansible.cfg
```

2.2 Inventory & connectivity

```
ansible all -i hosts.ini -m ping
ansible webserver -i hosts.ini -a "uptime"
ansible all --list-hosts
ansible-inventory -i hosts.ini --graph
```

2.3 Ad-hoc commands

```
ansible all -m shell -a "df -h"
ansible all -m copy -a "src=file.txt dest=/tmp/file.txt"
ansible all -m apt -a "name=nginx state=present" -b
ansible all -m service -a "name=nginx state=started enabled=yes" -b
ansible all -m user -a "name=deploy state=present" -b
ansible all -m setup
```

2.4 Running playbooks

```
ansible-playbook site.yml
ansible-playbook site.yml --check # dry run
ansible-playbook site.yml --syntax-check
ansible-playbook site.yml --limit webserver
ansible-playbook site.yml --tags "install,config"
ansible-playbook site.yml -e "env=production"
ansible-playbook site.yml -vvv # verbose debugging
```

2.5 Roles & Galaxy

```
ansible-galaxy init myrole
ansible-galaxy install geerlingguy.nginx
ansible-galaxy collection install community.docker
```

2.6 Vault (secrets management)

```
ansible-vault create secrets.yml
ansible-vault encrypt vars.yml
ansible-vault edit secrets.yml
ansible-playbook site.yml --ask-vault-pass
ansible-playbook site.yml --vault-password-file .vault_pass.txt
```

2.7 Debugging & facts

```
ansible all -m setup -a "filter=ansible_distribution*"
ansible-playbook site.yml --step # confirm each task
```

```
ansible-playbook site.yml --start-at-task="Install nginx"
```

3. Interview Questions — Application & Scenario Based

These go beyond definitions — they're the kind of applied, scenario-based questions asked in real DevOps/SRE interviews.

Q1. A playbook needs to restart a service only if a config file actually changed. How do you implement this, and why does it matter?

Use a **handler** triggered by `notify` on the task that changes the config file (e.g. the `template` or `copy` task). Handlers only fire when the notifying task reports `changed: true`, and they run once at the end of the play even if notified multiple times. This avoids unnecessary restarts (and downtime) on every run — a core idempotency pattern.

Q2. You need to run a playbook against 500 servers, but only 3 are currently down. How do you prevent the whole run from failing?

Set `any_errors_fatal: false` (the default) so Ansible continues with the remaining hosts; failed hosts are automatically added to a retry file (`*.retry`). You can also tune `max_fail_percentage` to abort only if failures exceed a threshold, and use `serial` for rolling batches so a bad batch is caught before it hits all hosts.

Q3. How would you deploy to 200 servers without taking the whole fleet down at once?

Use `serial` in the play (e.g. `serial: 10%` or `serial: [5, 20, '100%']` for a staged rollout), combined with health checks (`wait_for`, `uri` module) between batches and `max_fail_percentage` to halt on early failures — this implements a rolling deployment.

Q4. Two engineers both hardcode passwords in a playbook committed to Git. What's wrong, and how does Ansible solve it?

Secrets in plain text in version control are a security risk. Ansible Vault encrypts variable files (`ansible-vault encrypt secrets.yml`) so they can be safely committed; the vault password itself is kept out of Git (env var, password manager, or CI secret store) and supplied at run time with `--vault-password-file` or `--ask-vault-pass`.

Q5. A task works fine when run alone but the second run of the whole playbook shows unexpected 'changed' status. What's likely wrong, and how do you fix it?

The task probably isn't idempotent — e.g. using `shell/command` to append to a file instead of using a proper module (`lineinfile`, `template`, `copy`). Fix by replacing shell hacks with the matching declarative module, or adding `creates/changed_when` conditions to `shell/command` tasks so Ansible can correctly detect state.

Q6. How do you structure variables so that a QA environment and a production environment use different values, without duplicating playbooks?

Keep one set of playbooks/roles and separate **inventories per environment** (or one inventory with groups), each with its own `group_vars/qa.yml` and `group_vars/production.yml`. Ansible

merges variables by precedence (role defaults < group_vars < host_vars < extra-vars), so the same playbook picks up the right values automatically based on which inventory/group you target.

Q7. You need the same nginx role reused across 10 different projects with slightly different settings. How do you design that?

Build it as a proper role with sensible values in `defaults/main.yml` (easily overridden) and only put values that shouldn't change in `vars/main.yml`. Each consuming playbook then overrides only what it needs via play vars, group_vars, or role parameters — keeping the role itself untouched and reusable (this mirrors how Ansible Galaxy roles are designed).

Q8. During a deploy, Ansible needs to pull the current git commit hash and pass it into a container tag. How do you do this within a playbook?

Use the `command` or `git` module to fetch it, register the result with `register: commit_hash`, then reference `{{ commit_hash.stdout }}` in a later task (e.g. as a Docker image tag). This shows the register/fact-passing pattern used constantly in real pipelines.

Q9. Your Terraform apply creates new EC2 instances with dynamic IPs. How does Ansible know which servers to configure?

Use a **dynamic inventory** — either the `amazon.aws.aws_ec2` inventory plugin that queries AWS directly by tag, or have Terraform's `local-exec` provisioner write the new IPs into a generated inventory file that Ansible then consumes. This removes the need to hand-maintain static inventory files as infrastructure changes.

Q10. A junior engineer suggests writing all logic in one giant playbook. Why is that a bad idea, and what's the alternative?

A monolithic playbook becomes hard to test, reuse, and reason about, and increases blast radius for mistakes. The alternative is breaking logic into **roles** (one responsibility each — webserver, database, monitoring), composed together by a thin top-level playbook (`site.yml`) using `roles:` or `import_playbook`. This mirrors good software engineering practice: single responsibility, reusability, and easier testing per module.

Q11. How would you avoid Ansible making a change and immediately reporting success even though the underlying service actually failed to start?

Don't rely purely on the module's own 'changed/ok' status for critical services — follow up with a verification task, e.g. `wait_for` on the port, or a `uri` module health-check request, and use `failed_when/assert` to fail the play explicitly if the real-world check doesn't pass. Never assume 'module returned ok' equals 'service is healthy'.

Q12. How do you test a playbook change safely before running it against production?

Run with `--check` (dry-run, no changes applied) plus `--diff` to preview file changes, validate syntax with `--syntax-check`, and ideally run the full playbook first against a staging inventory that mirrors production. Tools like Molecule can also automate role testing in disposable Docker containers as part of CI.

End of guide — pair this with the hands-on modules in the [Ansible-Learning-Hub](#) repository.